

Лабораторная работа №5

Артём Дмитриевич Петлин

2026-04-18

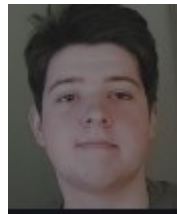
Содержание I

- 1 Информация
- 2 Цель работы
- 3 Задание
- 4 Теоретическое введение
- 5 Выполнение лабораторной работы
- 6 Выводы

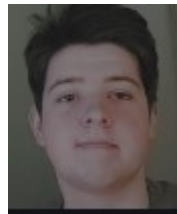
Раздел 1

Информация

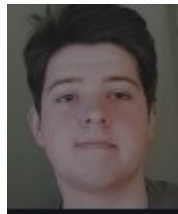
- Петлин Артём Дмитриевич



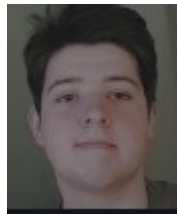
- Петлин Артём Дмитриевич
- студент



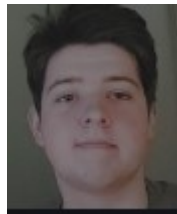
- Петлин Артём Дмитриевич
- студент
- группа НПИбд-02-24



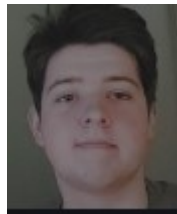
- Петлин Артём Дмитриевич
- студент
- группа НПИбд-02-24
- Российский университет дружбы народов



- Петлин Артём Дмитриевич
- студент
- группа НПИбд-02-24
- Российский университет дружбы народов
- 1132246846@pfur.ru



- Петлин Артём Дмитриевич
- студент
- группа НПИбд-02-24
- Российский университет дружбы народов
- 1132246846@pfur.ru
- https://github.com/hikrim/study_2025-2026_infosec-intro



Раздел 2

Цель работы

Цель работы

Изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов.
Получение практических навыков работы в консоли с дополнительными атрибутами.
Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

Раздел 3

Задание

- Создание программы

- Создание программы
- Исследование Sticky-бита

Раздел 4

Теоретическое введение

Помимо прав администратора для выполнения части заданий потребуются средства разработки приложений. В частности, при подготовке стенда следует убедиться, что в системе установлен компилятор gcc (для этого, например, можно ввести команду `gcc -v`).

Если же gcc не установлен, то его необходимо установить, например, командой

```
yum install gcc
```

которая определит зависимости и установит следующие пакеты: gcc, cloog-ppl, cpp, glibc-devel, glibc-headers, kernel-headers, libgomp, ppl, cloog-ppl, cpp, gcc, glibc-devel, glibc-headers, kernel-headers, libgomp, libstdc++-devel, mpfr, ppl, glibc, glibc-common, libgcc, libstdc++.

Раздел 5

Выполнение лабораторной работы

Входим в систему от имени пользователя guest. Создаём программу simpleid.c с кодом для вывода эффективных UID и GID. Компилируем программу и убеждаемся, что файл создан.



The image shows a terminal window at the top with the following commands and output:

```
$ gcc simpleid.c -o simpleid
$ ls
Documents Downloads Music Pictures Public simpleid simpleid.c Templates Videos
$
```

Below the terminal is a code editor window titled 'simpleid.c' showing the following C code:

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t uid = geteuid ();
    gid_t gid = getegid ();
    printf ("uid=%d, gid=%d\n", uid, gid);
    return 0;
}
```

Рисунок 1: simpleid

```
guest@adpetlin:~$ ./simpleid
uid=1001, gid=1001
guest@adpetlin:~$ id
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Рисунок 2: simpleid | id

Выполняем программу simpleid. Выполняем системную программу id и сравниваем результаты. Обе команды показывают одинаковые идентификаторы пользователя guest, так как программа запущена без специальных битов.

Усложняем программу, добавляя вывод действительных идентификаторов (`real_uid`, `real_gid`), сохраняем как `simpleid2.c`.
Компилируем и запускаем `simpleid2`.

```
guest@adpetlin:~$ gcc simpleid2.c -o simpleid2
guest@adpetlin:~$ ls
Desktop  dir1  Documents  Downloads  Music  Pictures  Public  simpleid  simpleid2  simpleid2.c  Template
guest@adpetlin:~$ ./simpleid2
e_uid=1001, e_gid=1001
real_uid=1001, real_gid=1001
guest@adpetlin:~$
```



```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main()
{
    uid_t real_uid = getuid();
    uid_t e_uid = geteuid();
    gid_t real_gid = getgid();
    gid_t e_gid = getegid();
    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
    printf ("real_uid=%d, real_gid=%d\n", real_uid, real_gid);
    return 0;
}
```

Рисунок 3: `simpleid2`

От имени суперпользователя выполняем команды. Первая команда меняет владельца файла на root и группу на guest, вторая устанавливает SetUID-бит, что позволяет программе выполняться с правами владельца файла (root), а не пользователя, запустившего её. Проверяем правильность установки атрибутов. В выводе видим букву „s“ на месте исполнительного бита владельца, что подтверждает установку SetUID-бита. Владелец файла — root. Запускаем simpleid2 и id, сравниваем результаты.

```
guest@adpetlin:~$ su -
Password:
Last login: Sat Apr 18 11:46:12 MSK 2026 on pts/0
root@adpetlin:~# chown root:guest /home/guest/simpleid2
root@adpetlin:~# chmod u+s /home/guest/simpleid2
root@adpetlin:~# exit
logout
guest@adpetlin:~$ ls -l simpleid2
-rwsr-xr-x. 1 root guest 16920 Apr 18 11:50 simpleid2
guest@adpetlin:~$ ./simpleid2
e_uid=0, e_gid=1001
real_uid=1001, real_gid=1001
guest@adpetlin:~$ id
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
guest@adpetlin:~$
```

Рисунок 4: simpleid2 | id

Создаём программу readfile.c для чтения файлов. Компилируем её. Меняем владельца файла и права так, чтобы только root мог читать его. Проверяем, что пользователь guest не может прочитать файл.

```
guest@adpetlin:~$ gcc readfile.c -o readfile
guest@adpetlin:~$ su -
Password:
Last login: Sat Apr 18 11:55:57 MSK 2026 on pts/0
root@adpetlin:~# chown root:root /home/guest/readfile.c
root@adpetlin:~# chmod 600 /home/guest/readfile.c
root@adpetlin:~# exit
logout
guest@adpetlin:~$
guest@adpetlin:~$ cat readfile.c
cat: readfile.c: Permission denied
guest@adpetlin:~$
```

Рисунок 5: readfile

```
guest@adpetlin:~$ su -  
Password:  
Last login: Sat Apr 18 11:57:35 MSK 2026 on pts/0  
root@adpetlin:~# chown root:root /home/guest/readfile  
root@adpetlin:~# chmod u+s /home/guest/readfile
```

Рисунок 6: setUID-бит

Устанавливаем setUID-бит.

Проверяем, может ли программа `readfile` прочитать файлы `readfile.c` и `/etc/shadow`? Да, программа может прочитать файл `readfile.c`.
Программа может прочитать `/etc/shadow`.

```
guest@adpetlin:~$ ./readfile readfile.c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;
    int fd = open (argv[1], O_RDONLY);
    do
    {
        bytes_read = read (fd, buffer, sizeof (buffer));
        for (i = 0; i < bytes_read; ++i) printf("%c", buffer[i]);
    }
    while (bytes_read == sizeof (buffer));
    close (fd);
    return 0;
}
guest@adpetlin:~$ ./readfile /etc/shadow
root:$y$j9T$Pol.Xh.rkags1VY0LXLvqtQ3$nnC47fN7ueb0wfI795Emlvm73IZNMebEiN5WXepu04A::0:99999:7:::
bin:*:20186:0:99999:7:::
daemon:*:20186:0:99999:7:::
adm:*:20186:0:99999:7:::
lp:*:20186:0:99999:7:::
sync:*:20186:0:99999:7:::
shutdown:*:20186:0:99999:7:::
halt:*:20186:0:99999:7:::
mail:*:20186:0:99999:7:::
operator:*:20186:0:99999:7:::
games:*:20186:0:99999:7:::
ftp:*:20186:0:99999:7:::
nobody:*:20186:0:99999:7:::
```



```
guest@adpetlin:~$ ls -l / | grep tmp
drwxrwxrwt. 17 root root 4096 Apr 18 11:59 tmp
guest@adpetlin:~$ echo "test" > /tmp/file01.txt
guest@adpetlin:~$ ls -l /tmp/file01.txt
-rw-r--r--. 1 guest guest 5 Apr 18 12:02 /tmp/file01.txt
guest@adpetlin:~$ chmod o+rw /tmp/file01.txt
guest@adpetlin:~$ ls -l /tmp/file01.txt
-rw-r--rw-. 1 guest guest 5 Apr 18 12:02 /tmp/file01.txt
guest@adpetlin:~$
```

Рисунок 8: /tmp

Выясняем, установлен ли атрибут Sticky на директории /tmp. Он установлен. Просматриваем атрибуты файла и разрешаем чтение и запись для «остальных».

От пользователя guest2 пробуем прочитать файл. Чтение успешно. От пользователя guest2 пробуем дозаписать в файл слово test2. Дозапись должна быть успешна (у меня какая-то ошибка). Проверяем содержимое файла. От пользователя guest2 пробуем записать слово test3, стирая содержимое. Операция снова должна быть успешна. Проверяем содержимое: должно было быть «test3». От пользователя guest2 пробуем удалить файл, у меня получилось, что удаление возможно, но это так не должно быть.

```
guest2@adpetlin:~$ cat /tmp/file01.txt
test
guest2@adpetlin:~$ echo "test2" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
guest2@adpetlin:~$ cat /tmp/file01.txt
test
guest2@adpetlin:~$ echo "test3" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
guest2@adpetlin:~$ cat /tmp/file01.txt
test
guest2@adpetlin:~$ rm /tmp/file01.txt
rm: cannot remove '/tmp/file01.txt': No such file or directory
guest2@adpetlin:~$ rm /tmp/file01.txt
rm: cannot remove '/tmp/file01.txt': No such file or directory
guest2@adpetlin:~$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? n
guest2@adpetlin:~$
```

Рисунок 9: guest2

Повышаем права до суперпользователя и снимаем Sticky-бит с /tmp. От пользователя guest2 проверяем отсутствие атрибута t. После снятия Sticky-бита пользователь guest2 может удалить файл /tmp/file01.txt.

```
guest2@adpetlin:~$ su -
Password:
Last login: Sat Apr 18 11:58:51 MSK 2026 on pts/0
root@adpetlin:~# chmod -t /tmp
root@adpetlin:~# exit
logout
guest2@adpetlin:~$ ls -l / | grep tmp
drwxrwxrwx. 19 root root 4096 Apr 18 12:05 tmp
guest2@adpetlin:~$ cat /tmp/file01.txt
test
guest2@adpetlin:~$ echo "test2" >> /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
guest2@adpetlin:~$ echo "test3" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
guest2@adpetlin:~$ rm /tmp/file01.txt
rm: cannot remove '/tmp/file01.txt': No such file or directory
guest2@adpetlin:~$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? n
guest2@adpetlin:~$
```

Рисунок 10: chmod -t /tmp

```
guest2@adpetlin:~$ su -  
Password:  
Last login: Sat Apr 18 12:07:30 MSK 2026 on pts/0  
root@adpetlin:~# chmod +t /tmp  
root@adpetlin:~# exit  
logout  
guest2@adpetlin:~$
```

Рисунок 11: chmod +t /tmp

Возвращаем атрибут t на директорию /tmp: chmod +t /tmp

Раздел 6

Выводы

Мы изучили механизмы изменения идентификаторов, применения SetUID- и Sticky-битов. Мы получили практические навыки работы в консоли с дополнительными атрибутами. Мы рассмотрели работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

Раздел 7

Список литературы

Список литературы